Basic Networking on Proxmox

2026-10-17

NOTE: the notes say that the speaker is Chris almost all the time. This is because the only mic being used is the one in Chris's laptop.git

AI-generated content in notes may be incorrect. Learn more Decisions

- Configure VM CPU topology with three sockets, one core each, and an eight-CPU limit.
- Create a pre-network snapshot and avoid deleting snapshots until lock issues are resolved.
- Configure static VM networking with the assigned address, gateway, DNS, and proxy.
- Configure APT proxy access, update package metadata, and install cowsay and fortune. Open question
- Resolve Network Manager/Netplan conflicts so static settings persist after reboot. Agenda

Meeting notes Meeting Documentation

- Chris said the previous week's AI-generated meeting notes had been overly verbose, so they planned to evaluate the built-in facilitator's notes for this meeting instead Virtual Machine Resources
- Chris changed the VM processor configuration to three sockets with one core each. The change did not appear to affect CPU consumption, which generally remained at one CPU, but the VM seemed faster and more responsive
- Isaiah's VM was shown as having four CPUs, although Chris did not establish whether Isaiah was currently running an installation or other CPU-intensive work inside it
- Chris compared a VM configuration with one socket and four cores and said it appeared more responsive based on his experience configuring the system the previous year. He also set a CPU limit of eight, although it was not confirmed whether this would dynamically provide additional CPU capacity under heavier load
- Chris advised checking the VM hardware configuration after shutdown, including turning **NUMA off** and leaving the CPU setting as **host**; he also confirmed that retaining **8 GB of memory** was acceptable SPICE Viewer Setup
- Chris explained that the SPICE Viewer could be installed through a documented `winget install --id` command. He noted that the package ID needed the correct capitalization and that adding `--exact` could cause the command to fail if the ID capitalization was incorrect
- Chris confirmed that the SPICE viewer was using a white interface and discussed whether its panels could be resized or hidden; one panel could be made smaller, but a full-hide option for the other was not identified
- Chris noted that the SPICE viewer could be switched to full-screen mode because remaining inside the viewer window could cause unusual resolution behavior Virtual Machine Operations
- Isaac had fixed the unexpected error encountered while executing the command, and the participants confirmed that their virtual machines were functioning and could be shut down and restarted successfully
- Chris explained that a VM could be shut down from the desktop by selecting the power icon, choosing **Power Off**, and confirming the shutdown if a console-session warning appeared. The VM was considered fully stopped once its status showed **guest not running** or the stop button became disabled

- Before configuring networking, Chris recommended taking a VM snapshot so the system could be restored instead of reinstalled if the configuration failed. Snapshot names could not contain spaces; he used **before network setup** as the example name, and said completion was indicated by a disabled stop button or **task OK** status
- After the snapshot completed, Chris instructed participants to start their VMs again, noting that the snapshot provided a recovery point if the VM was accidentally powered off or the configuration caused problems
- Chris and the participants encountered VM lock errors while starting VMs 113 and 117, including an inability to connect after 51 retries and start operations that timed out and failed. The issue appeared related to lock files that could remain after deleting a VM and potentially block other VM operations
- Chris explained that Iowa State had previously advised contacting them to delete an unnecessary lock file when VM deletion caused locking problems. The participants suspected the same lock-file issue was affecting the rollback and start attempts
- A rollback of VM 113 completed, but attempts to start the affected VMs still failed or left the controls grayed out. Chris also noted that accidentally deleting a snapshot might have contributed to the problem, and the participants agreed not to delete snapshots during class until the issue was understood
- A previously failing VM began booting successfully after the related lock file was deleted, although another VM remained slow to start and was still at approximately 55% during login Configuration Naming
- The group preferred using an explicit vendor name in the configuration to avoid ambiguity with another item using the same name within the namespace Text Input Tools
- Chris explained that PowerToys Quick Accent could be enabled with a selectable activation key and used to insert accented characters while typing. They used it at work to represent Romanian teammates' names and city names accurately Network Fundamentals
- Chris explained that a DHCP server assigned IP addresses to devices, a DNS server converted computer or domain names into IP addresses, and a router forwarded network packets to other networks such as the internet
- Chris explained that an address such as 192.168.1.123 on the 192.168.1.0/24 network used the first three number groups, 192.168.1, to identify the network. Devices on that same network could communicate without sending traffic through the router Ubuntu Operations
- Chris showed that Ubuntu terminals could be opened with **Ctrl+Alt+T** or launched from the dash after pinning the terminal application there, which provided a convenient way to reopen terminals
- Chris emphasized explicitly saving configuration changes before exiting nano because relying on defaults had previously caused troubleshooting problems; they recommended writing the file immediately after making a significant edit rather than waiting until exit
- Chris explained that on the server configuration, non-privileged users could not reboot through the System V mechanism and would receive an error unless the reboot request used an interactive prompt with elevated privileges. Ubuntu's default desktop installation still allowed rebooting or powering off through the graphical power menu
- Chris introduced Ubuntu's two primary package-management systems, Snap and Apt. They explained that Apt served as a front end for `dpkg`, the Debian package manager, while Aptitude was a higher-level front end for Apt
- Chris explained that updating packages with Apt required two commands and root privileges. For the demonstration, they entered `sudo -i` to open a root shell; a password prompt appeared only if the

user had not authenticated recently, and a successful transition displayed a root prompt containing the host name

- Chris explained that Apt maintained local metadata about packages and their dependencies rather than storing the packages themselves. When installing a program, Apt used this dependency information to identify and retrieve the additional packages required for the program to function, such as `libc`, which Chris described as a necessary dependency for Linux platforms
- Chris clarified that `apt update` refreshed Apt's package metadata, while `apt upgrade` installed available package updates and did not automatically run `apt update`; both commands were therefore required when the package information needed to be refreshed before upgrading
- Chris demonstrated that `apt upgrade` could prompt for confirmation before continuing, and the upgrade was aborted by answering "no" when the participants only wanted to inspect or install a specific package
- Chris showed how to use `apt search` to locate packages and explained that search results could include related variants such as a graphical `xcowsay` package. He identified `cowsay` as the package to install with `apt install cowsay` and noted that package results included the target platform and version information
- Chris introduced the `fortune` package as an additional command-line utility and demonstrated searching for it with `apt search fortune`. The search returned multiple related results, so the participants reviewed the package descriptions to identify the correct package rather than selecting an unrelated variant
- Chris demonstrated that `apt install` displayed the additional packages and dependencies that would be installed before confirmation. For the example, installing GEdit would also install packages such as `gedit-common`, GLib, GTK, and related dependencies, allowing users to review the installation before accepting it
- Chris explained that the `fortune` command was installed under `/usr/games`, which was not available on the current shell path. They temporarily added `/usr/games` to `PATH` with `PATH=/usr/games:$PATH`, after which running `fortune` produced randomized quotations
- Chris used the `fortune` output as input for `cowsay`, demonstrating that the utility could display a message such as "I am a real cow" in an ASCII cow speech bubble Network Diagnostics
- Chris demonstrated that the `ip` command referred to Internet Protocol and was used to inspect or work with IP addresses as part of the networking stack
- Chris explained that the default route acted as the default gateway: traffic destined for anything outside the local 192.168.1 network was sent to the gateway address, which then forwarded it to other networks
- Chris demonstrated that pinging the default gateway succeeded when the machine had obtained an IP address through DHCP. They noted that a missing or unreachable gateway would prevent external connectivity and could produce a network-unreachable error
- Chris explained that DNS was not yet working reliably in the environment. A request using a hostname timed out, while accessing HTTPS://1.1.1.1 by IP address returned a response, showing that direct IP connectivity was available even though name resolution was not
- Chris explained that `whois` was commonly used with domain names but could also identify the organization assigned a particular IP address block. For example, the lookup showed that the `160.153/16` block belonged to GoDaddy, while another lookup identified the RIPE Network Coordination Centre and provided an abuse-reporting contact
- The environment already contained the correct proxy configuration, but the machine could reach its default gateway at 199.116.100 while failing to reach the proxy. A telnet test to the proxy's port 3128 also failed, so outbound internet access through the proxy was not working

- After the proxy configuration, Chris tested connectivity with `wget 1.1.1.1` without relying on DNS, but the request failed with a "no route to host" error. They then investigated shell execution and found that the environment was still not fully configured
- During proxy testing, a malformed URL that omitted the `http://` protocol and slash was corrected after the connection attempt failed; the subsequent test still returned a **"no route to host"** result, so outbound connectivity through the proxy remained unresolved
- During route troubleshooting, Chris used the `ip` command to inspect the routing table and questioned whether the configured gateway was actually the VM's default gateway. They noted that the proxy-related settings were being handled separately and suggested consulting NetworkManager documentation for the correct network configuration rather than relying only on manually entered values
- A direct `wget` test to `1.1.1.1` worked temporarily when the VM had the expected gateway, but Chris expected connectivity to fail again after reboot because the active settings appeared to be coming from Netplan
- When Chris stopped or disabled the relevant network-management service during the experiment, the VM lost its network configuration and the network settings disappeared from the interface. This indicated that the service relationship or an underlying configuration still needed to be understood before changing the default service state Network Security
- Chris explained that machines in the playground and competition environments could not connect directly to the public internet because they were isolated from the real internet. A proxy was required for outbound internet access, helping protect the internal systems from attackers during the competition
- Chris noted that the network environment would include participants defending their systems while attackers attempted to access them, so the restriction on direct internet access was intended to reduce exposure of the internal environment rather than protect the internal systems from accessing the internet
- Chris explained that the playground and competition environments reproduced portions of the public internet, including GoDaddy address ranges, but isolated those environments from direct internet access so participants could not accidentally attack real organizations such as GoDaddy
- Chris explained that Iowa State placed a proxy between the competition network and the public internet. The proxy acted as an intermediary that received requests from the internal environment, inspected packets to verify that they were safe and legitimate, and relayed approved requests externally
- Chris said the School 14 environment was isolated on the `144.76.90/24` network, whose gateway was the `.1` address, and that the gateway was the only route out of the isolated network Web Infrastructure
- Chris noted that proxies were commonly placed in front of web servers and identified NGINX as a specific example of proxy software
- Chris explained that a proxy forwarded client traffic to another destination and could perform safety checks and caching to improve connection speed. NGINX was given as an example of software placed in front of web servers, while Cloudflare was cited as another large-scale proxy service Proxy Configuration
- Chris said the system proxy needed to be configured for the environment, rather than setting a proxy only for an individual user, because the current setup was not providing connectivity
- Chris configured manual DNS using the value documented as `199.116.100`, explaining that the DNS server was the same as the proxy address. He searched the CDC 2026 documentation because some DNS information from the previous year had been incorrect

- Chris explained that the manual network configuration was necessary because DHCP would not assign the specific address required by the VM, and that the system proxy also needed to be configured after the IP address, gateway, and DNS settings
- Chris discussed that proxy setup could be applied through scripts on Linux and Windows, but the environment required several separate settings, including environment variables and proxy configuration. He decided to test a system-wide configuration through the network GUI instead of relying only on the script-based approach
- Chris located the Ubuntu desktop proxy setup documentation, which included the proxy, gateway, and network-mask values, and noted that the documented GUI procedure would be used to configure the connection
- Chris configured the proxy entries with the same proxy URL, `199.100.16.100`, and port `3128` for each proxy setting. They noted that the URL worked without an explicit protocol header and saved the configuration through the system interface
- Chris identified `/etc/wgetrc` as another relevant configuration file and noted that it contained a `use_proxy = yes` setting. They concluded that additional proxy entries might be required before `wget` could use the proxy successfully
- Chris instructed participants to edit `/etc/wgetrc` by uncommenting all three proxy entries, setting each URL to `http://199.100.16.100:3128/`, and enabling `use_proxy = yes`; they saved the file in nano with **Ctrl+O**, retained the existing filename, confirmed the write, and then exited with **Ctrl+X**
- Chris noted that the relevant configuration contained three proxy settings and that all of them needed to use the same proxy address and port, even though one comment suggested that an explicit setting might not be required
- Chris demonstrated that Apt required proxy entries in its configuration, using `Acquire::http::Proxy` and `Acquire::https::Proxy` with the environment's proxy address and port `3128`. They emphasized that the proxy URL syntax, including the protocol, quotation marks, and terminating semicolon, had to be entered exactly because these were common sources of configuration errors Network Configuration
- Chris instructed participants to configure each VM with a manual IPv4 address rather than DHCP. The address used the `144.76.90` network followed by the VM ID, such as `144.76.90.117`, with a `/24` network prefix and a gateway using the same network address ending in `.1`
- Chris explained that the system-wide "Make available to other users" setting needed to be enabled for the wired connection. He noted that server configurations would differ because servers would not use the same graphical interface and would instead use Netplan
- Chris confirmed that the network configuration could use automatic routes, a `/24` network mask, and the documented proxy and gateway values. He also identified `resolvectl` as the command for controlling or checking DNS resolution and confirmed that it detected the configured DNS settings
- Chris identified that the VM's manually entered network settings did not persist after reboot: the terminal temporarily used the new configuration, but the graphical network settings reverted to automatic. They suspected that a Netplan configuration was overriding the NetworkManager settings
- Chris reviewed the manual network values for the `144.76.90` network, including a `/24` prefix, the VM address ending in `117`, gateway `144.76.90.254`, and DNS value `199.116.100`. They clarified that routes should remain automatic and should not be manually changed
- Chris investigated the relationship between Netplan and NetworkManager, locating NetworkManager-related configuration under `/etc/NetworkManager` and noting that the relevant service could be inspected or disabled with `systemctl` if it was overriding the intended network configuration
- Chris explained that the NetworkManager service was enabled to start automatically at boot and was currently running, but it conflicted with Netplan because both attempted to control the network

configuration. NetworkManager appeared to take precedence when both were enabled

- Chris demonstrated that `systemctl disable` prevented the NetworkManager service from starting automatically, while `systemctl stop` halted the service immediately; disabling it alone did not stop a service that was already running. The service status also showed that it had been manually disabled even though its default preset was enabled
- Chris re-enabled NetworkManager after the networking experiment, retained a temporarily working address, and confirmed that the browser could access and download internet content. This indicated that connectivity was restored at that point, although the interaction between NetworkManager and Netplan remained relevant to the configuration